EXPERIMENT-08

Aim

To design and implement an automated, serverless Resume Portfolio Website on AWS that publishes newly uploaded resumes and documents to a public website, using Amazon S3 for storage and hosting, AWS Lambda for automatic processing, Amazon CloudWatch for centralized log monitoring, and IAM for access control.

Software / Tools Required

- Amazon Web Services (AWS) account with S3, Lambda, IAM, and CloudWatch access
- Two Amazon S3 buckets (private upload bucket and public website bucket)
- AWS Lambda with Python 3.12 runtime and Boto3 library
- HTML, CSS, and JavaScript for the website front end
- Web browser for testing the deployed website

Theory

A Resume Portfolio Website is a serverless web application designed to store, publish, and share professional documents such as resumes, certificates, and project reports, without the need to maintain a traditional web server. In this experiment, the system is built entirely using AWS managed services, each responsible for a specific function within the overall pipeline.

Amazon S3 forms the core of the storage and hosting layer. Two separate buckets are used: a private bucket that receives newly uploaded documents, and a public bucket that hosts the static website and stores the documents made available to visitors. Separating the buckets in this way ensures that raw uploads remain inaccessible to the public until they have been processed and published.

AWS Lambda provides the automation layer. Whenever a new object is created in the private upload bucket, an S3 event notification automatically invokes the Lambda function, without requiring any manual intervention. The function retrieves the uploaded document, copies it into the website bucket, and updates a JSON file that acts as the master index of published documents.

AWS IAM enforces the principle of least privilege. An IAM role attached to the Lambda function grants only the specific S3 permissions required to read from the upload bucket and to read and write within the website bucket, while a managed execution policy allows the function to write its logs to CloudWatch.

Amazon CloudWatch provides observability for the system. Every execution of the Lambda function, along with any errors that may occur, is logged to a dedicated CloudWatch log group, allowing the upload-to-publish pipeline to be monitored and verified after the fact.

Finally, the front end of the website is implemented using HTML, CSS, and JavaScript. It dynamically reads the document index from the JSON file and provides visitors with search and download functionality, along with a custom error page for any invalid page requests.

Procedure

Step 1: Open AWS Console

- Open the AWS Management Console.
- Sign in to your AWS account.
- Select the region Asia Pacific (Mumbai).

- Search for S3 from the AWS services menu.

Step 2: Create the Website S3 Bucket

- Open Amazon S3 and click Create bucket.
- Enter the bucket name: resume-site-raviraj
- Select the AWS region as Asia Pacific (Mumbai).
- Under Block Public Access settings, disable Block all public access, and confirm the warning.
- Keep the remaining settings as default and click Create bucket.
- This bucket is used for hosting the Resume Portfolio Website.

Step 3: Create the Private Upload Bucket

- Click Create bucket again and enter: resume-upload-raviraj
- Select Asia Pacific (Mumbai).
- Keep Block all public access enabled.
- Keep the remaining settings as default and click Create bucket.
- This bucket is used to upload resumes and other documents. The final bucket structure is:
- resume-site-raviraj
- resume-upload-raviraj

Step 4: Create Website Files

- Create the following three files on the computer: index.html, resumes.json, error.html
  4.1 index.html — Contains the Resume Portfolio interface: website title, document list, search box, download buttons, and JavaScript for loading resumes.json. The website reads document names from resumes.json and provides download links from documents/.
  4.2 resumes.json — Initially created with an empty list []; this file is later updated automatically by Lambda.
  4.3 error.html — A simple custom 404 page containing “404 / Page Not Found / Go Home”, displayed when a requested webpage does not exist.

Step 5: Upload Website Files to S3

- Open the bucket resume-site-raviraj and click Upload.
- Add index.html, resumes.json, and error.html, then click Upload.
- The website bucket now contains the basic website files.

Step 6: Enable Static Website Hosting

- Open resume-site-raviraj and go to the Properties tab.
- Find Static website hosting and click Edit.
- Enable static website hosting and select Host a static website.
- Enter Index document: index.html and Error document: error.html, then save.
- AWS provides a website endpoint similar to
  http://resume-site-raviraj.s3-website.ap-south-1.amazonaws.com. The exact endpoint displayed by AWS should be used.

Step 7: Configure S3 Bucket Policy

- The website bucket must allow visitors to read website files.
- Open resume-site-raviraj, go to Permissions, and find Bucket policy.
- Add a policy that allows public s3:GetObject access to objects in the website bucket. The resource is:
- arn:aws:s3:::resume-site-raviraj/*
- Save the policy.

- This allows visitors to access the static website files. The upload bucket remains private.

Step 8: Create IAM Policy

- Search for IAM, open it, select Policies, and click Create policy.
- Select the required S3 permissions. The Lambda function needs permission to read uploaded files from resume-upload-raviraj, read and update resumes.json, and write published documents into the website bucket. The important resources are:
- arn:aws:s3:::resume-upload-raviraj/*
- arn:aws:s3:::resume-site-raviraj/resumes.json
- arn:aws:s3:::resume-site-raviraj/documents/*
- Give the policy a suitable name, for example Resume-Lambda-Policy, and create it.

Step 9: Create IAM Role for Lambda

- In IAM, open Roles and click Create role.
- Select AWS service, then select Lambda as the use case.
- Attach Resume-Lambda-Policy and also attach AWSLambdaBasicExecutionRole.
- Give the role a name such as Resume-Lambda-Role and create it.
- This role gives Lambda permission to perform the required S3 operations and write logs to CloudWatch.

Step 10: Create Lambda Function

- Open AWS Lambda, click Create function, and select Author from scratch.
- Enter Function name: resume-publisher and select the Python 3.12 runtime.
- Under permissions, select Use an existing role, choose Resume-Lambda-Role, and click Create function.

Step 11: Configure Lambda Environment Variable

- Inside the Lambda function, go to Configuration > Environment variables.
- Add Key: SITE_BUCKET, Value: resume-site-raviraj, and save.
- This tells Lambda which S3 bucket contains the website.

Step 12: Add Lambda Code

- Open the Code section of the Lambda function and enter the program that performs the following operations: receives the S3 event, reads the uploaded filename, gets the uploaded document from the private bucket, copies it to resume-site-raviraj/documents/, reads resumes.json, adds the new document name, updates resumes.json, and prints execution information to CloudWatch. The processing flow is:
- Upload document -> S3 Event -> Lambda -> Copy document -> Update resumes.json -> Website displays document
- Click Deploy after entering the code.

Step 13: Configure S3 Trigger

- Open the Lambda function resume-publisher and click Add trigger.
- Select S3 and choose the bucket resume-upload-raviraj.
- Select event type s3:ObjectCreated:*, confirm the configuration, and click Add.
- The trigger should now appear in the Lambda function, so it runs automatically on every upload.

Step 14: Upload a Resume/Document

- Open Amazon S3 and open resume-upload-raviraj.

- Click Upload and select a resume, certificate, or project document, for example raviraj cert.pdf.
- Click Upload.
- The upload bucket is private, so visitors cannot directly access this bucket.

Step 15: Verify Lambda Execution

- Open AWS Lambda and open resume-publisher.
- Check the Monitor section and open the CloudWatch logs.
- The logs should show messages similar to: “S3 event received”, “Received file: raviraj cert.pdf”, “Published: raviraj cert.pdf”, and “resumes.json updated”. This confirms that the Lambda function was automatically triggered.

Step 16: Verify Published Document

- Open the S3 bucket resume-site-raviraj and check for the folder documents/.
- Open the folder; the uploaded document should appear there.
- This confirms that Lambda successfully copied the document from the private upload bucket to the website bucket.

Step 17: Verify resumes.json

- Open resume-site-raviraj and check resumes.json.
- The file should no longer contain only []; it should contain the uploaded document name. This file acts as the document list used by the website.

Step 18: Open the Resume Portfolio Website

- Go to the S3 static website endpoint.
- The Resume Portfolio page should open, and the uploaded document should appear in the list.

Step 19: Test Search Functionality

- Open the website, click the search box, and enter part of the document name, for example cert.
- The website should filter the document list so that only matching documents are displayed.
- This confirms that the JavaScript search functionality is working.

Step 20: Test Download Functionality

- Find the uploaded document and click the Download button.
- The document should open or download in the browser.
- This confirms that the published document is accessible through the website.

Step 21: Test 404 Error Page

- Open the website endpoint and change the URL to a page that does not exist, then press Enter.
- The custom page should display “404 / Page Not Found / Go Home”.
- This confirms that the configured error.html page is working.

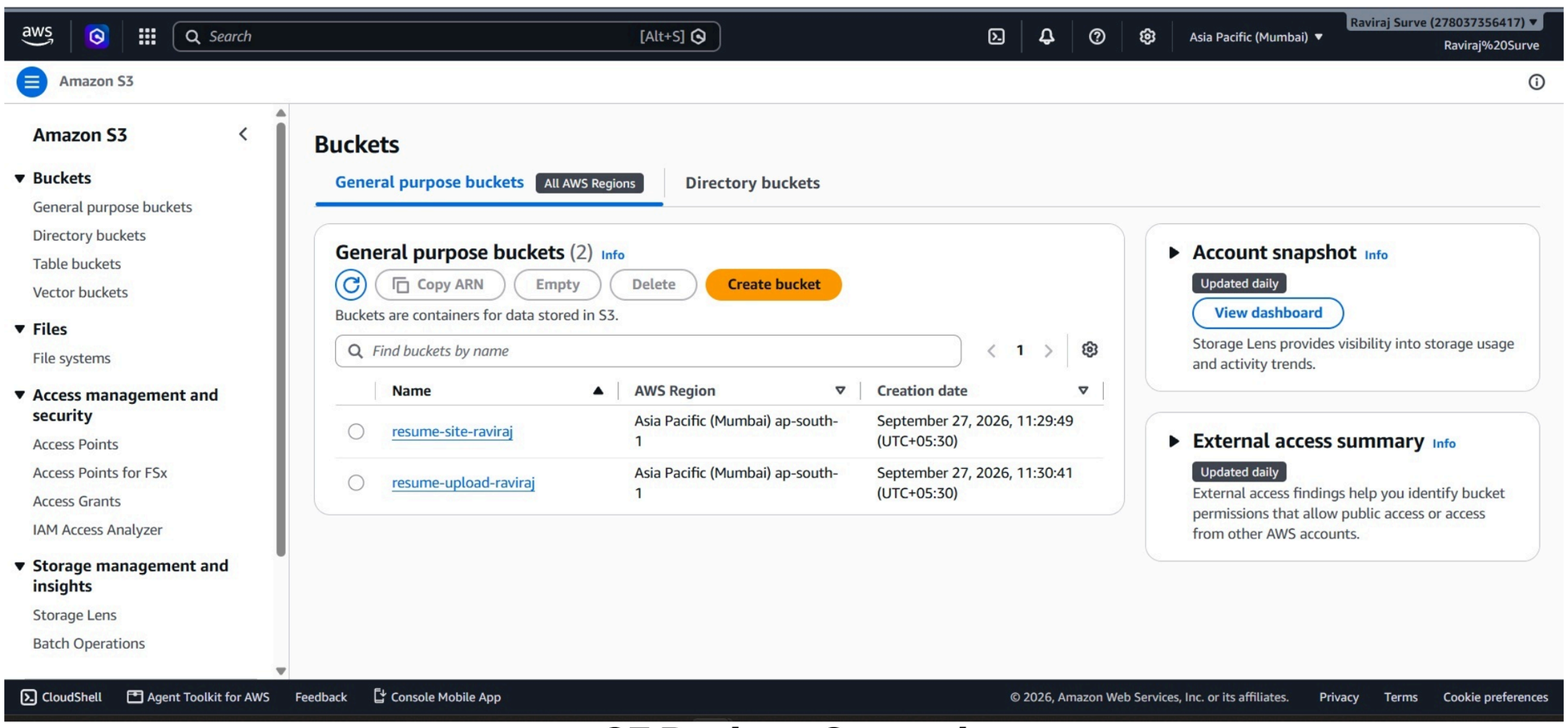
Step 22: Check CloudWatch Logs

- Open CloudWatch, select Logs, and open the Lambda log group /aws/lambda/resume-publisher.
- Open the latest log stream and verify that Lambda execution messages are present.
- The logs provide evidence that the S3 event was received, the file was received and published, and resumes.json was updated.

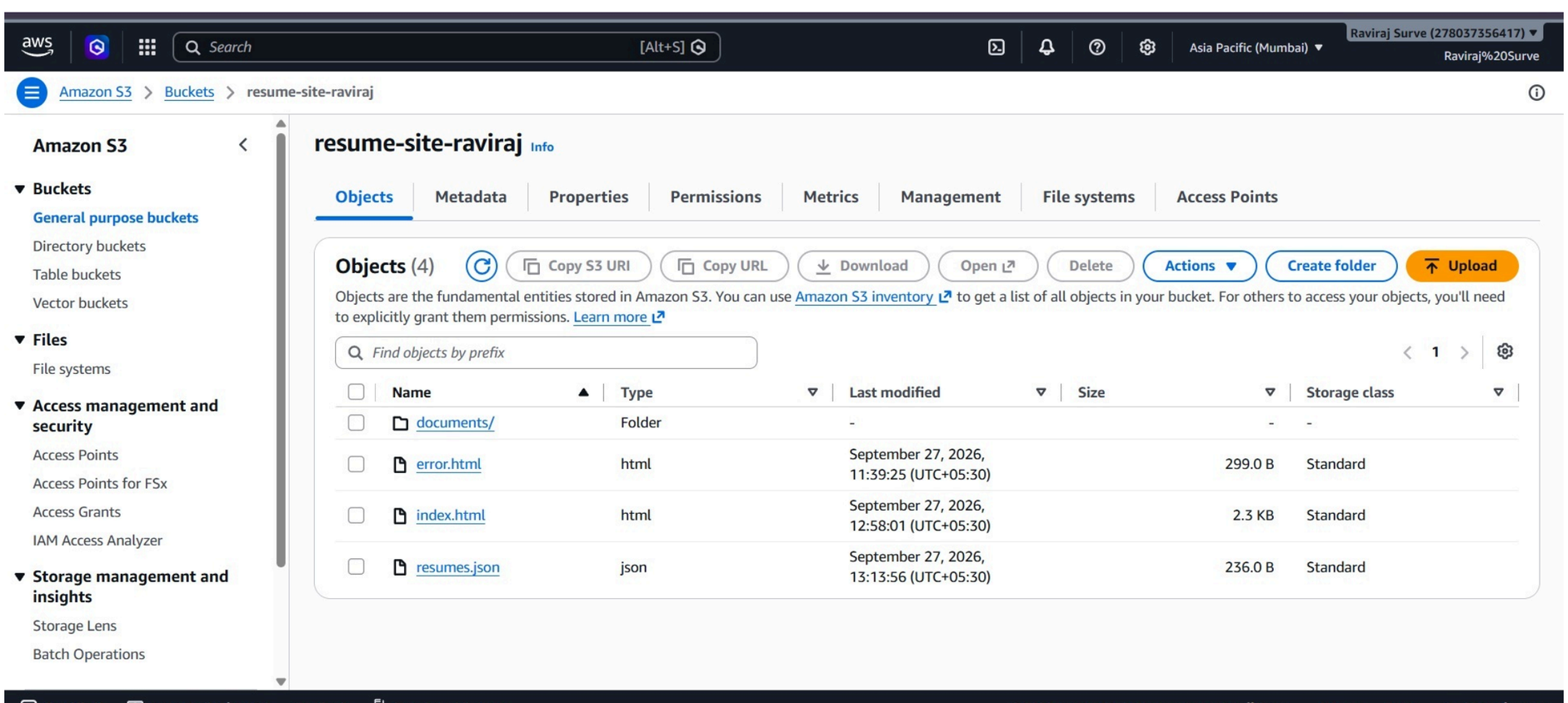
Step 23: Verify Complete Workflow

- Finally, verify the complete system:
- User -> Uploads resume -> resume-upload-raviraj -> S3 ObjectCreated Event -> resume-publisher Lambda -> Copy document -> Update resumes.json -> resume-site-raviraj -> Static Website -> Search / Download
- All components should work together automatically.

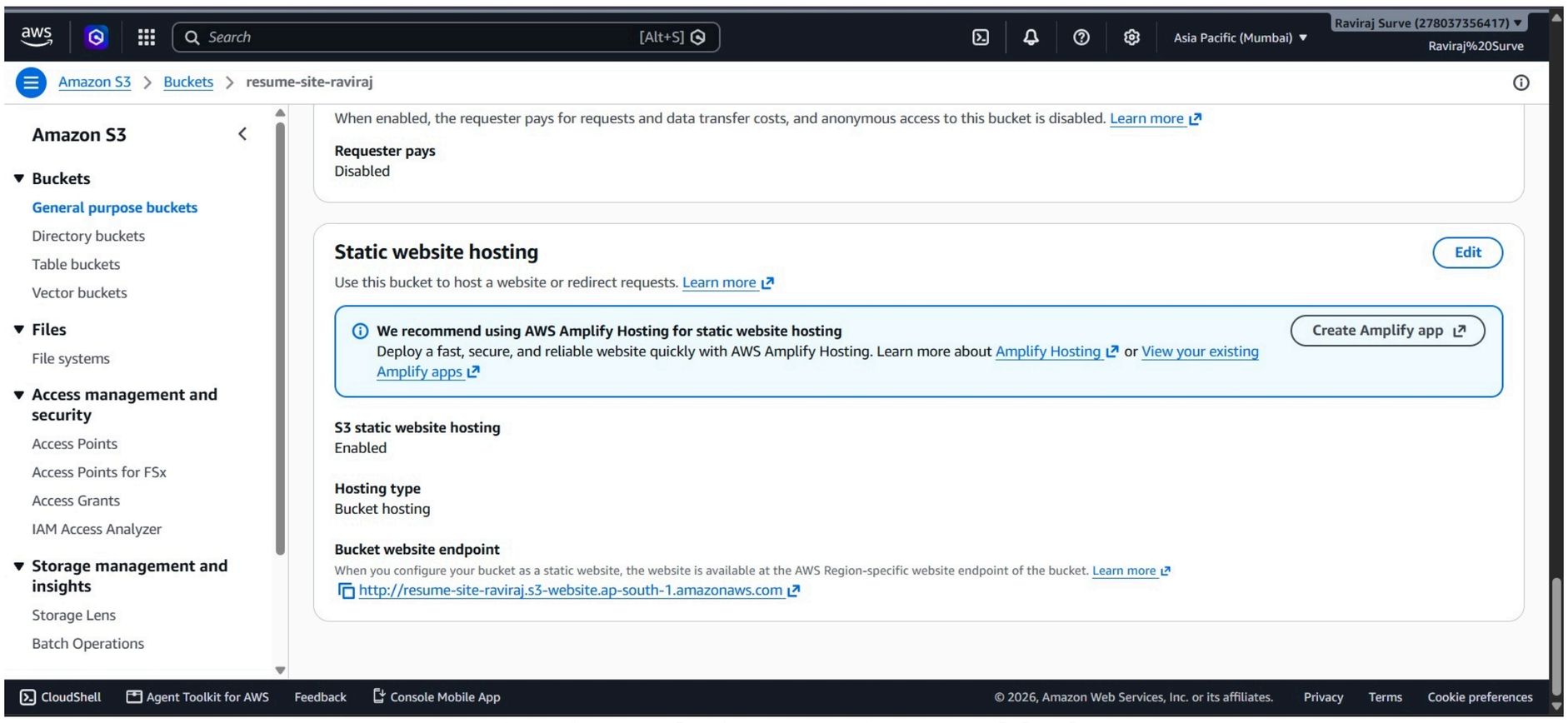
Output



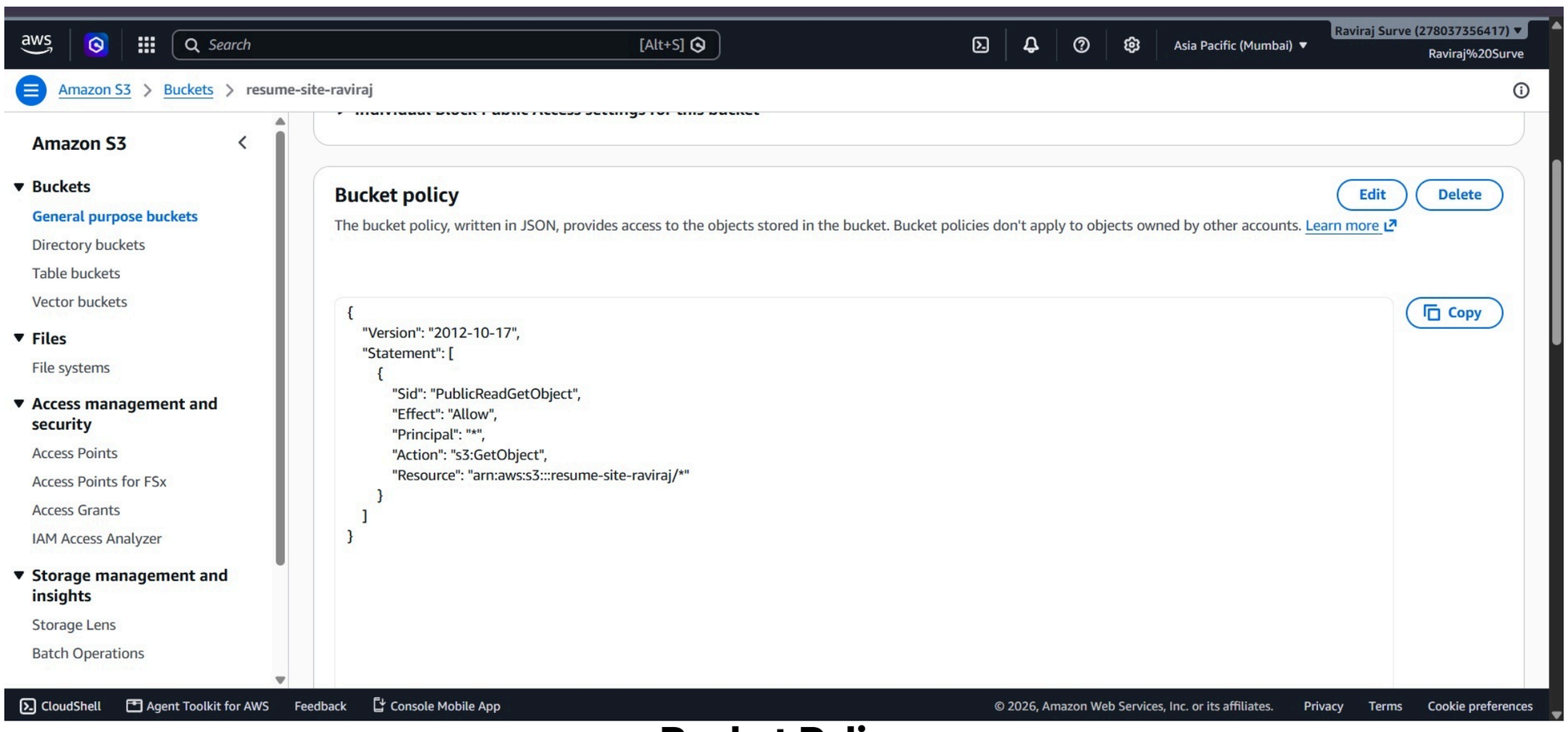
S3 Buckets Created



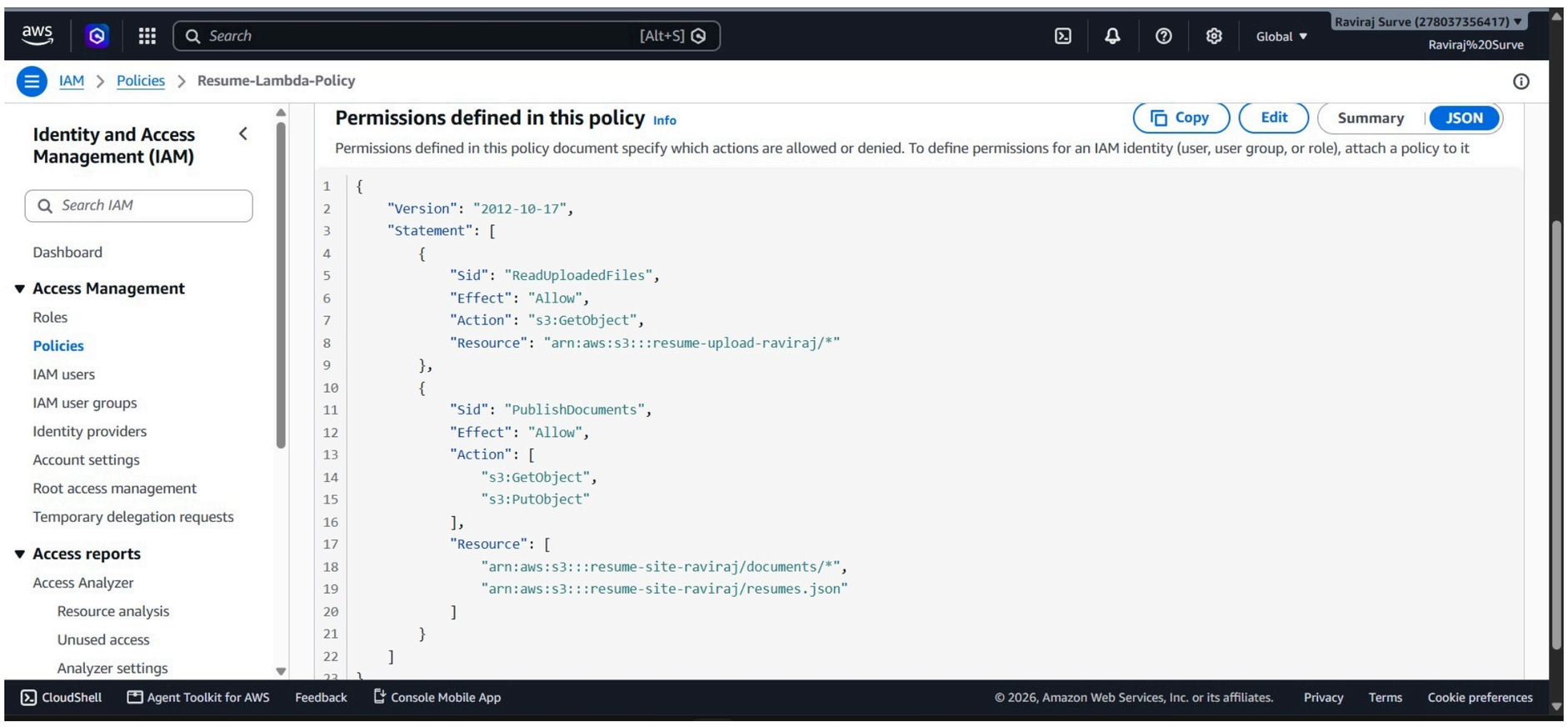
Website Files Uploaded



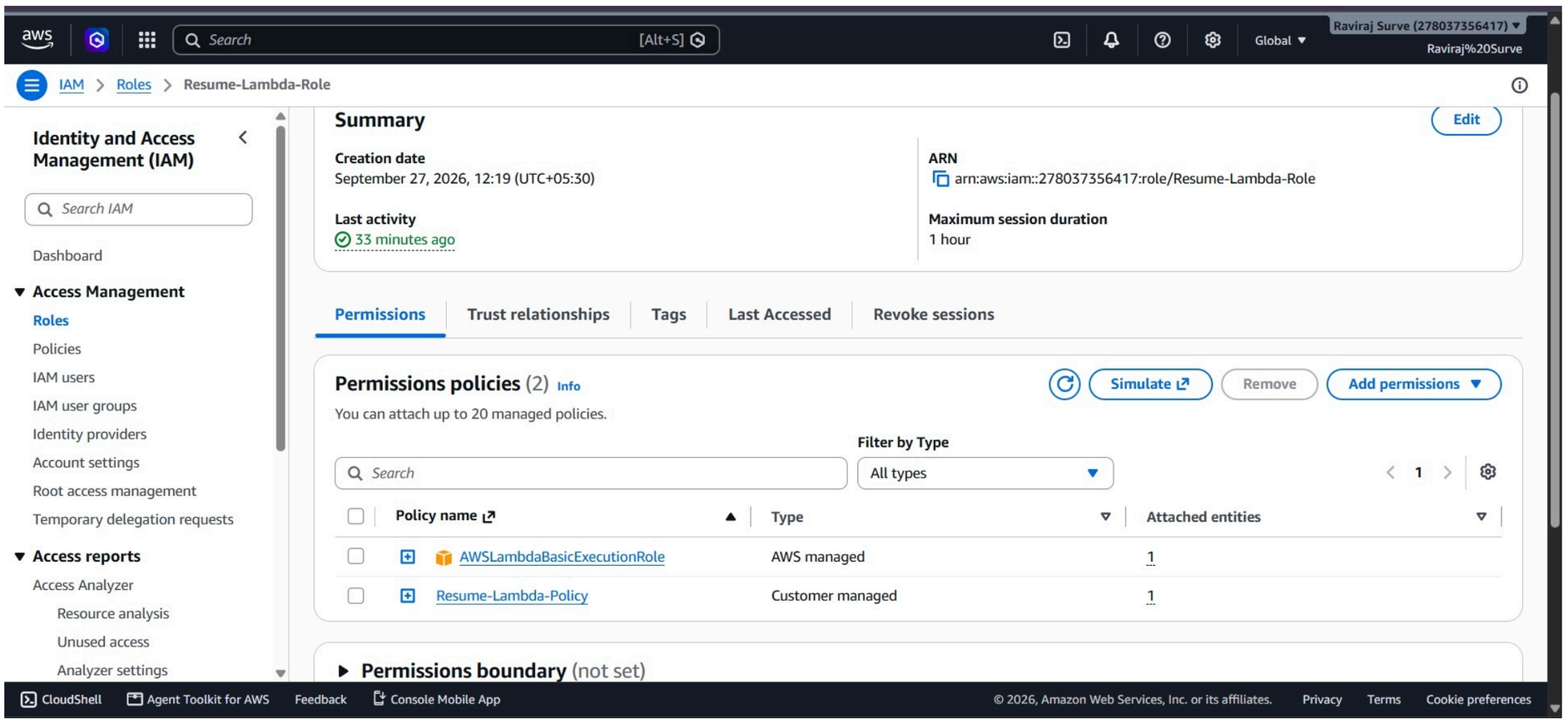
Static Website Hosting Enabled



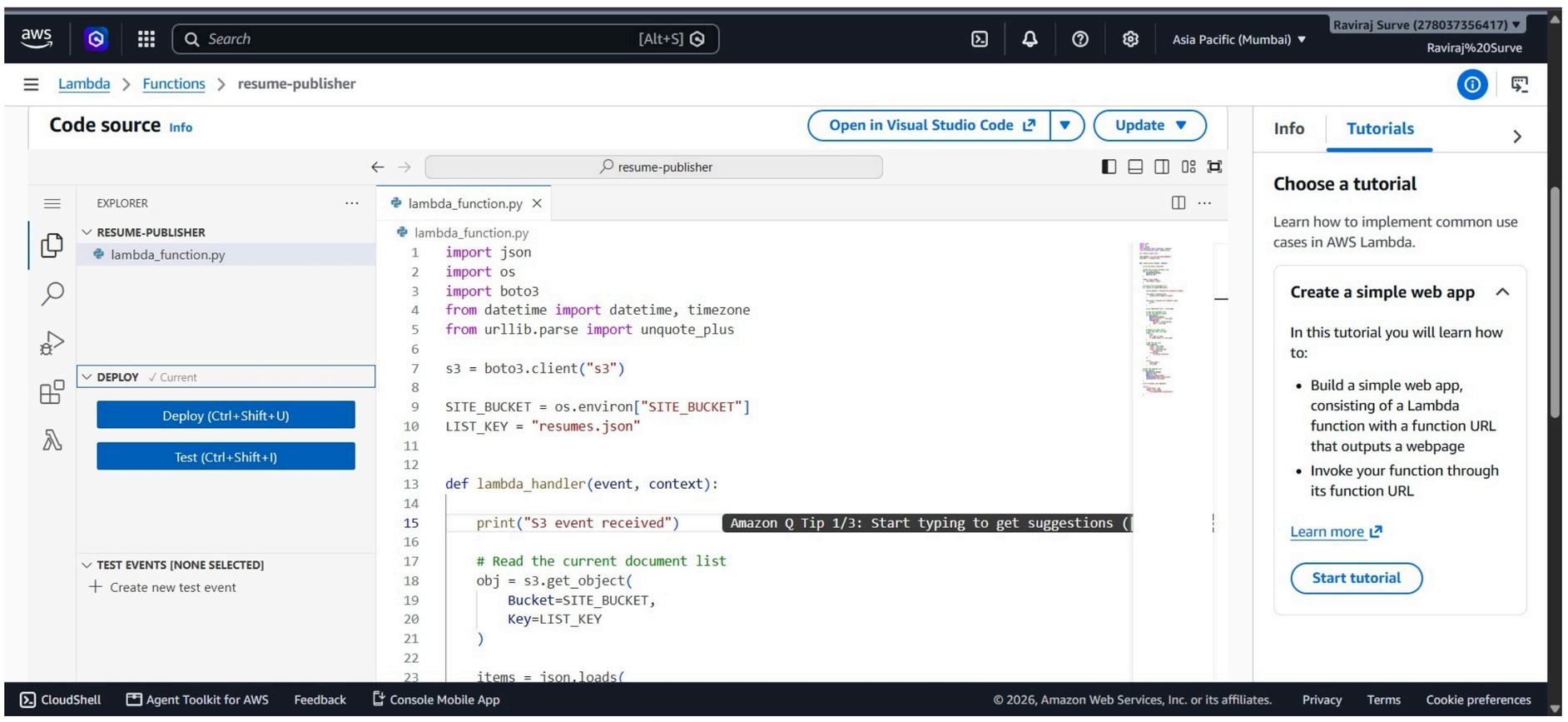
Bucket Policy



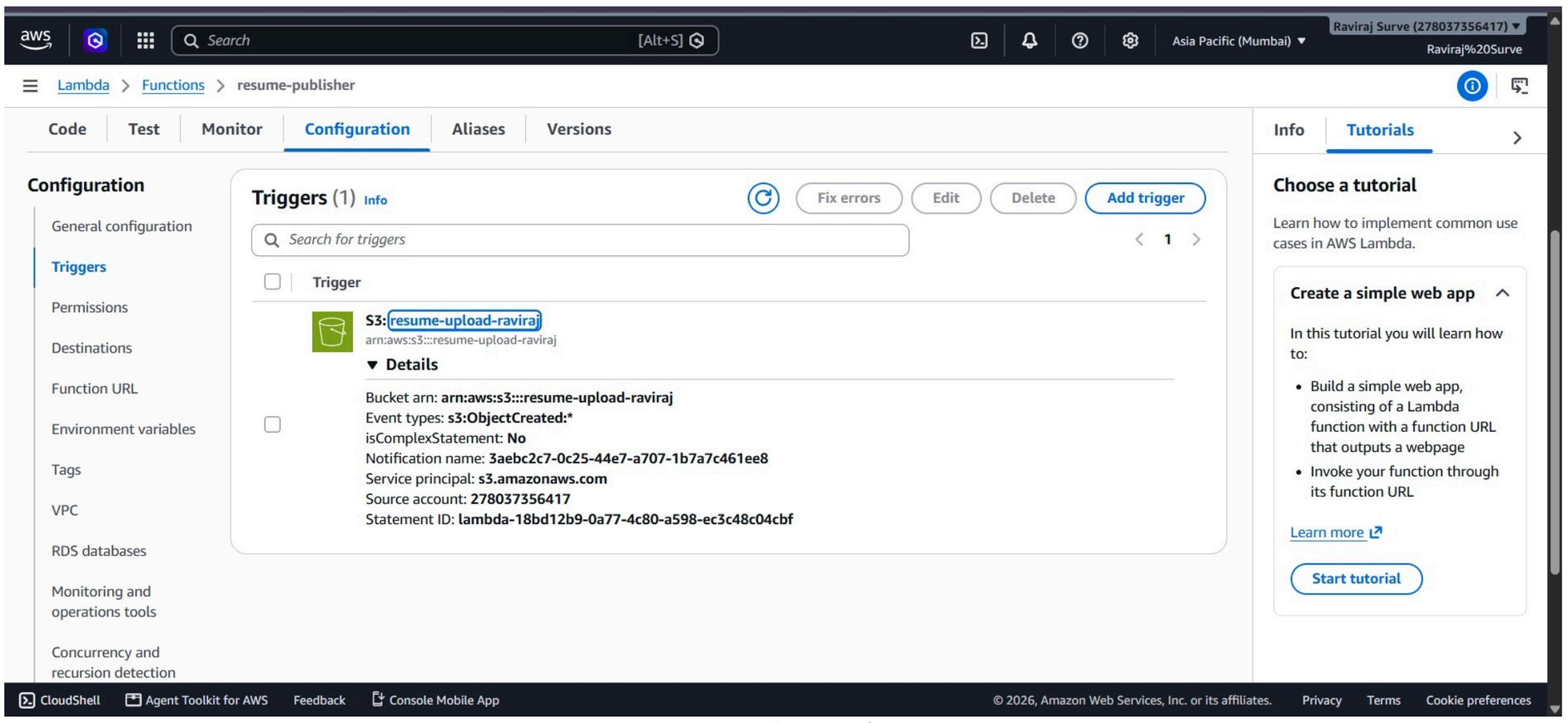
IAM Policy



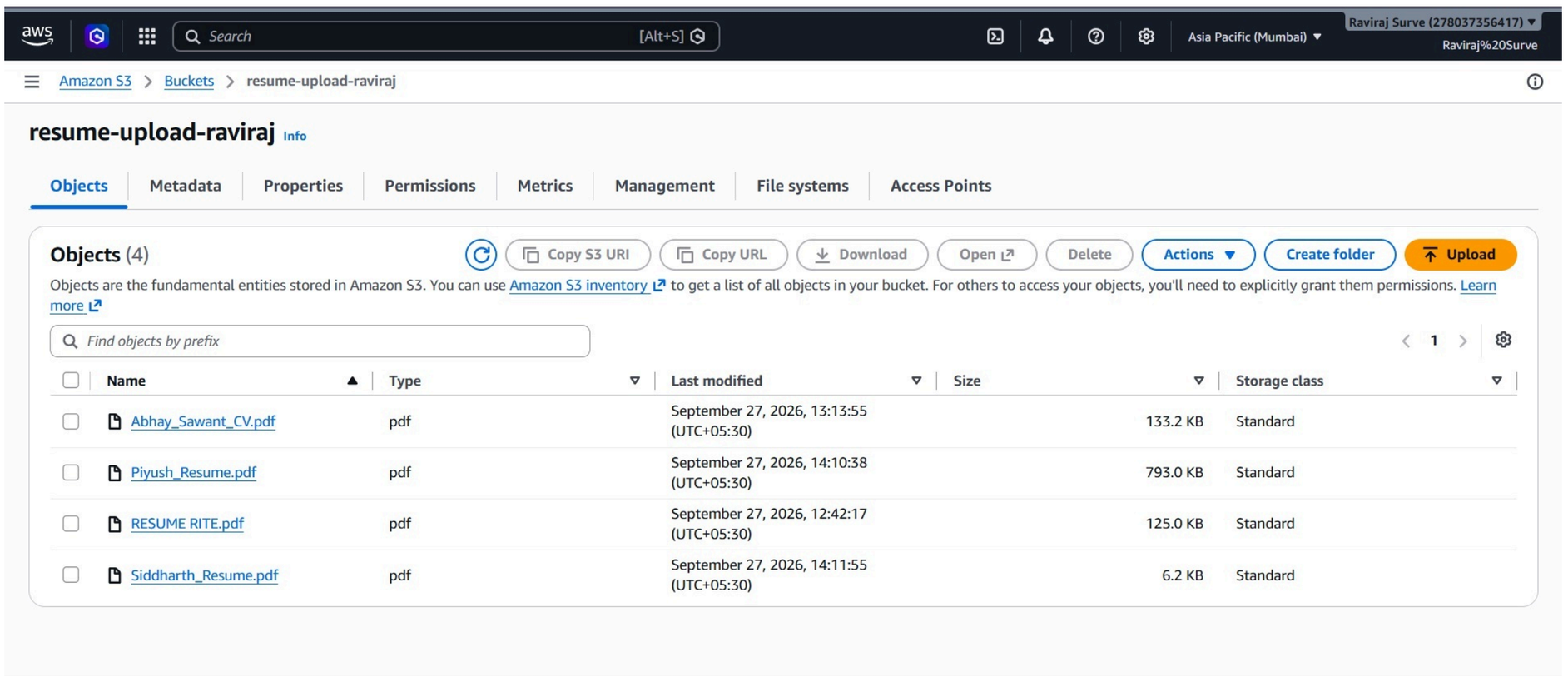
IAM Role



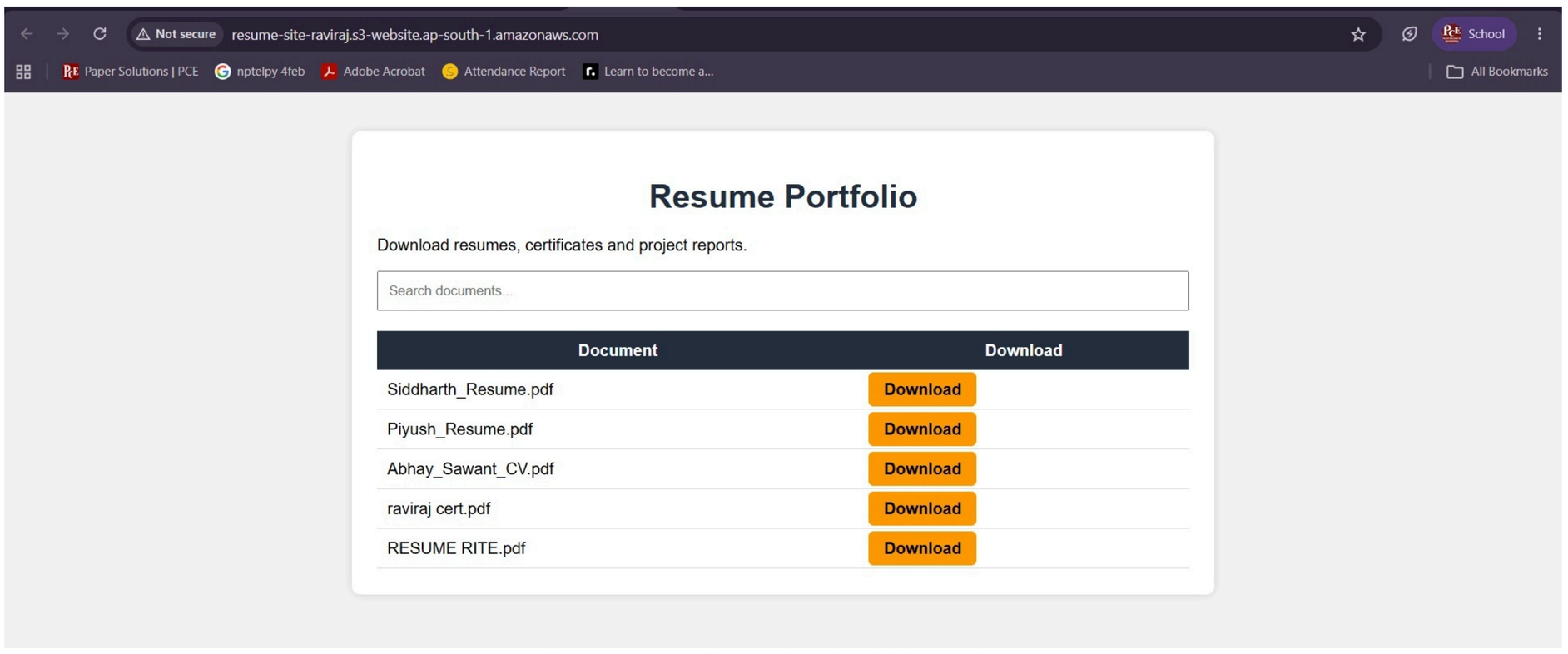
Lambda Function Code



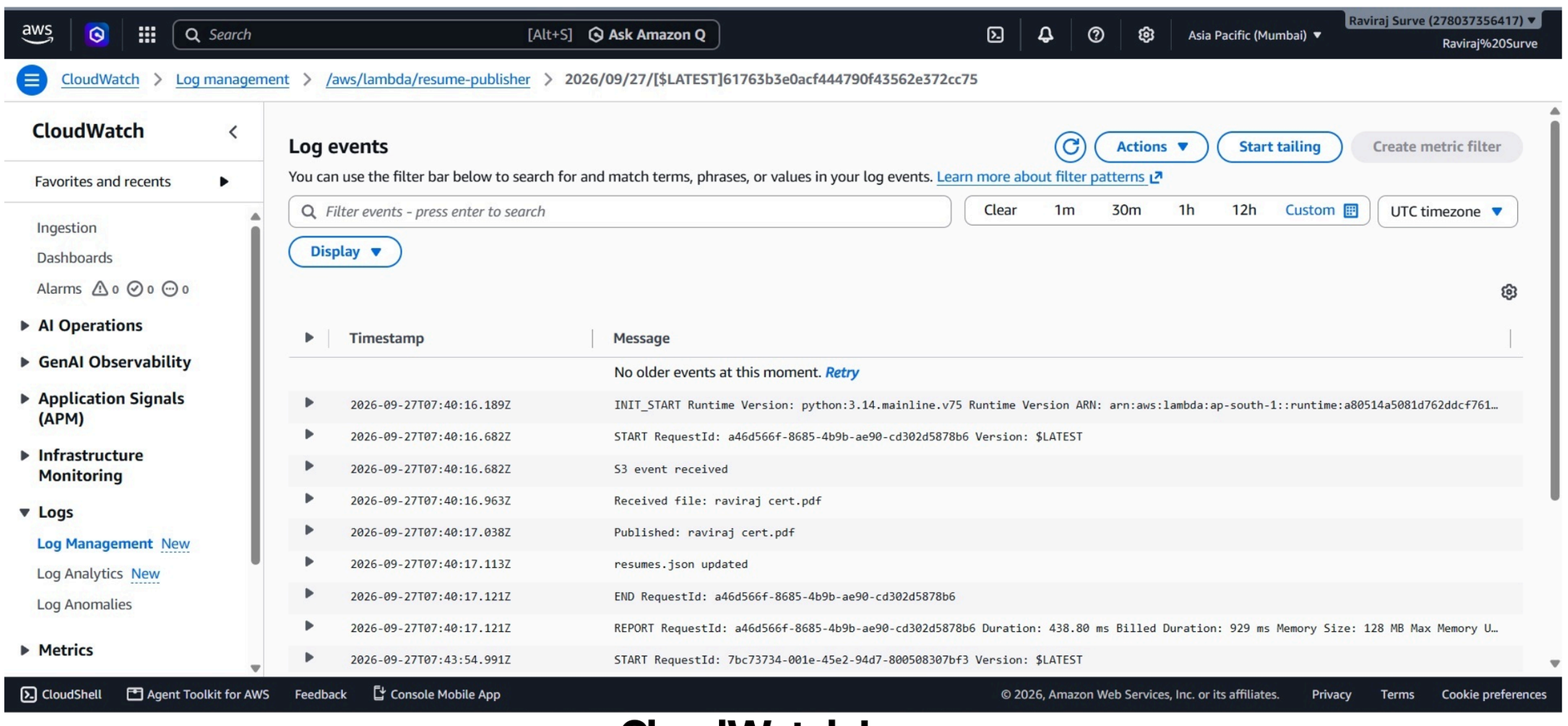
Lambda S3 Trigger



Document Uploaded to Private S3 Bucket



Resume Portfolio Website



CloudWatch Logs

Result

The Resume Portfolio Website was successfully deployed. Uploading a document to the private bucket automatically triggered the Lambda function, which published the file to the website bucket, updated resumes.json, and logged every step to CloudWatch. The live website correctly displayed, searched for, and served the uploaded document, and the custom 404 page worked as expected.

Conclusion

The automated Resume Portfolio Website was successfully implemented by integrating Amazon S3, AWS Lambda, IAM, and Amazon CloudWatch. The system eliminates manual publishing effort, automatically processes and indexes newly uploaded documents, and enables centralized, secure monitoring of application activity, thereby demonstrating a practical, automated, and access-controlled serverless web application built entirely on AWS.